Week Four: Automation and Webhooks

Portfolio evidence | Automation 01 and Flask webhook exercise

This document records the Week Four practical work: a three-step customer registration automation built in Make, and a separate Flask webhook endpoint developed from the Week Three application.

1. Business problem and purpose

A customer registration creates a small but repetitive administrative task. The registration details need to be recorded, and the customer should receive an acknowledgement. Completing these steps manually takes time and creates opportunities for information to be copied incorrectly or for a confirmation to be missed.

The Make exercise demonstrates how a webhook can receive a registration and pass the submitted details to connected services. The Flask exercise explores how an application can receive and validate structured data through its own HTTP endpoint. These are separate exercises; the Flask application is not connected to the Make scenario.

2. Make automation

Workflow

The scenario uses a webhook as its trigger, followed by two actions:

Step	Module	Purpose
1 — Trigger	Webhooks: Customer SignUp	Receives the customer registration data.
2 — Action	Google Sheets: Add a Row	Adds the submitted registration details to the spreadsheet.
3 — Action	Microsoft 365 Email: Send an Email	Sends a welcome email using the submitted customer details.

Workflow overview

CUSTOMER SIGN-UP WEBHOOK → GOOGLE SHEETS: ADD A ROW → OUTLOOK: SEND WELCOME EMAIL

Data received

The webhook receives customer registration information, including the customer's name, email address and company. The name and email fields are mapped into the subsequent actions.

Field	Description
name	Customer's name
email	Customer's email address
company	Company supplied with the registration

3. Flask webhook endpoint

The Week Three Flask application was extended with a POST endpoint at /hooks/signup. The endpoint accepts JSON registration data and validates the submitted fields before returning a response.

Request structure

POST /hooks/signup

```
{
  "name": "Jane Smith",
  "email": "jane.smith@example.com",
  "company": "Example Company"
}
```

Condition	Observed endpoint behaviour
Valid JSON registration	Returns HTTP 201 and a generated reference in the format CB-YYYYMMDD-NNNN.
Request is not JSON	Returns HTTP 415.
Required data is missing or invalid	Returns HTTP 400.
Email address is invalid	Returns HTTP 400.
Malformed or overlong input	Returns HTTP 400.

Testing

Five additional tests were added for the signup endpoint. The full Flask test suite completed successfully with 25 tests passing.

4. Data protection and responsible use

The Make test used synthetic customer information rather than real customer records. The workflow passes registration information between connected services, so an implementation handling real registrations would need to use appropriate access controls and ensure that only necessary personal information is shared.

5. Reflection

This week's work helped me understand the relationship between a trigger and the actions that follow it. The Make scenario made the sequence visible: information arrives through a webhook, is passed to a spreadsheet, and is then used to send a confirmation email. Seeing the successful execution recorded in Make History helped me understand how to check whether an automation had run.

The Flask work approached the same general idea from the application side. Rather than relying on a visual automation builder, I extended the existing application so that it could receive a structured request and check the information before responding. Writing tests for valid and invalid inputs reinforced the importance of testing more than the expected successful case.

The two exercises were useful in different ways. Make demonstrated how services can be connected through a workflow, while Flask gave me a clearer understanding of what happens when an application receives a webhook request.

6. Possible future development

A future version of the registration workflow could include additional checks for duplicate submissions, clearer handling of service errors, and notifications if an action fails. AI-assisted processing could also be explored where it adds a useful function, while keeping the handling of customer information appropriate and transparent.

7. Summary of evidence

Evidence	What it demonstrates
Make History screenshot	Submitted separately; shows the successful execution on 6 November 2026, with all three modules recorded as successful.
Flask endpoint	POST /hooks/signup accepts and validates JSON registration data.
Flask test results	25 tests passed across the full test suite.
Updated README	The project documentation was updated to reflect the Week Four work.

The Make scenario and Flask endpoint are documented as separate pieces of work. The document describes the behaviour implemented and tested during the Week Four exercises.